
CS771 Major Assignment 1: Breaking the Hamming PUF

Anany Bhardwaj
Roll No: 240121

Aditya Raunak
Roll No: 240067

Ketan
Roll No: 240539

Satvik Sharma
Roll No: 230936

Hardik Wadhwa
Roll No: 240423

Mehul Talwar
Roll No: 240651

Abstract

We present a method to break the Hamming PUF by constructing an explicit feature map that linearizes its quadratic decision boundary. The Hamming PUF computes the product of partial Hamming weights of even and odd bit positions after XOR with a secret key, then thresholds this product. We show that this operation, although involving multiplication and Hamming weights, can be perfectly captured by a 288-dimensional linear model through a suitable feature map consisting of the original 32 challenge bits and all 256 even-odd cross-product terms. Using LinearSVC with optimized hyperparameters, we achieve >99% test accuracy on the public dataset.

1 Mathematical Derivation of the Linear Model (Part 1)

Melbo's Hamming PUF utilizes a secret 32-bit word $s \in \{0, 1\}^{32}$ and calculates a response based on the product of the partial Hamming weights of the XOR masked challenge $m = c \oplus s$. To break this using a linear model, we must demonstrate that this non-linear sequence of operations can be mapped into a linearly separable feature space $\phi(c)$.

Expressing the XOR operation globally, for any challenge bit c_i and secret bit s_i , the XOR output m_i can be written as:

$$m_i = c_i \oplus s_i = c_i + s_i - 2c_i s_i = c_i(1 - 2s_i) + s_i \quad (1)$$

Since s_i is a constant for the PUF, we can define a constant weight $w_i = (1 - 2s_i) \in \{-1, +1\}$. Thus:

$$m_i = c_i \cdot w_i + s_i \quad (2)$$

Now we expand the partial hamming weights. Let $E = \{0, 2, 4, \dots, 30\}$ be the even indices and $O = \{1, 3, 5, \dots, 31\}$ be the odd indices. The partial Hamming weights are:

$$h_e = \sum_{i \in E} m_i = \sum_{i \in E} (c_i \cdot w_i + s_i) = \sum_{i \in E} c_i w_i + S_e \quad (3)$$

$$h_o = \sum_{j \in O} m_j = \sum_{j \in O} (c_j \cdot w_j + s_j) = \sum_{j \in O} c_j w_j + S_o \quad (4)$$

where $S_e = \sum_{i \in E} s_i$ and $S_o = \sum_{j \in O} s_j$ are PUF-specific constants.

Expanding the product $P = h_e \cdot h_o$

$$\begin{aligned}
P &= h_e \cdot h_o = \left(\sum_{i \in E} c_i w_i + S_e \right) \left(\sum_{j \in O} c_j w_j + S_o \right) \\
&= \underbrace{\sum_{i \in E} \sum_{j \in O} w_i w_j \cdot c_i c_j}_{256 \text{ cross-product terms}} + \underbrace{S_o \sum_{i \in E} w_i c_i + S_e \sum_{j \in O} w_j c_j}_{32 \text{ linear terms}} + \underbrace{S_e S_o}_{\text{constant}}
\end{aligned} \tag{5}$$

The last step we take is formulating the linear decision boundary. The response $r(c) = 1$ if $P \geq t$, else $r(c) = 0$. This is equivalent to:

$$r(c) = \frac{1 + \text{sign}(P - t)}{2} \tag{6}$$

From Eq. 5, P is a linear function of the features $\{c_i\}_{i=0}^{31}$ and $\{c_i c_j\}_{i \in E, j \in O}$. Therefore, the decision boundary $P - t = 0$ is a *hyperplane* in the 288-dimensional feature space:

$$\phi(c) = [c_0, c_1, \dots, c_{31}, c_0 c_1, c_0 c_3, \dots, c_{30} c_{31}]^\top \in \mathbb{R}^{288} \tag{7}$$

The corresponding linear model has:

- Weight vector $\mathbf{w} \in \mathbb{R}^{288}$, where the components corresponding to c_i have values $S_o w_i$ (for even i) or $S_e w_j$ (for odd j), and the components corresponding to $c_i c_j$ have values $w_i w_j$.
- Bias $b = S_e S_o - t$.

Since $r(c) = \frac{1 + \text{sign}(\mathbf{w}^\top \phi(c) + b)}{2}$, a linear model on $\phi(c)$ can perfectly predict all Hamming PUF responses.

2 Dimensionality of the Linear Model (Part 2)

The derived linear model requires a feature space that captures both the linear components and the cross product terms of the challenge bits. The dimensionality D is calculated as follows:

- **Linear Terms:** The 32 original challenge bits $(c_0, c_1, \dots, c_{31})$. This contributes **32** dimensions.
- **Cross-Product Terms:** The product of every even-indexed bit with every odd-indexed bit $(c_i \cdot c_j \text{ for } i \in E, j \in O)$. Since there are 16 even bits and 16 odd bits, there are $16 \times 16 = 256$ cross-product dimensions.

$$D = 32 + 256 = 288 \tag{8}$$

It is polynomial (quadratic) in the bit-length, not exponential, making it practical. The 288 features are linearly independent and each corresponds to a distinct monomial term in the expansion of $h_e \cdot h_o$.

3 Hyperparameter Experiments (Part 4)

To evaluate the stability and efficiency of our 288-dimensional mapping, we conducted experiments on the public dataset (7500 train, 2500 test) comparing different loss functions, regularization strengths, tolerance values, and penalty types for both LinearSVC and LogisticRegression.

3.1 Experiment A: Effect of Loss Function on LinearSVC

Table 1: Effect of loss function on LinearSVC (C=1.0, tol=10⁻⁴).

Loss Function	Dual	Training Time (s)	Train Acc (%)	Test Acc (%)
hinge	True	0.2767	99.88	99.72
squared_hinge	False	0.0532	99.96	99.68

Analysis: The squared_hinge loss with primal formulation (dual=False) is $5.2\times$ **faster** than hinge (dual), while achieving comparable accuracy. This is because squared_hinge is differentiable everywhere, enabling more efficient optimization. Both achieve $>99.6\%$ test accuracy, confirming that the 288-dimensional feature space is linearly separable.

3.2 Experiment B: Effect of Regularization Parameter C

Table 2: Effect of regularization parameter C on LinearSVC and LogisticRegression.

Model	C	Training Time (s)	Train Acc (%)	Test Acc (%)
LinearSVC	0.01	0.0422	98.56	97.40
LinearSVC	0.10	0.0587	99.56	98.72
LinearSVC	1.00	0.0688	99.96	99.68
LinearSVC	10.0	0.0904	100.00	99.60
LinearSVC	100.0	0.1066	100.00	99.56
LogisticRegression	0.01	0.0728	96.79	96.32
LogisticRegression	0.10	0.0677	98.57	97.72
LogisticRegression	1.00	0.1164	99.64	98.68
LogisticRegression	10.0	0.1281	100.00	99.32
LogisticRegression	100.0	0.2185	100.00	98.44

Analysis: Low regularization ($C = 0.01$) restricts the model’s ability to perfectly fit the linearly separable boundary, resulting in lower accuracy. Moderate values ($C = 1.0$) provide the best speed-accuracy tradeoff. Very high C (≥ 10) achieves 100% training accuracy but takes longer and can slightly overfit. LinearSVC consistently outperforms LogisticRegression in both speed and test accuracy, likely because the hinge-based loss is better suited for the perfectly separable data.

3.3 Experiment C: Effect of Tolerance

Table 3: Effect of convergence tolerance on LinearSVC and LogisticRegression ($C = 1.0$).

Model	tol	Training Time (s)	Train Acc (%)	Test Acc (%)
LinearSVC	10^{-2}	0.0473	99.87	99.36
LinearSVC	10^{-3}	0.0557	99.96	99.68
LinearSVC	10^{-4}	0.0665	99.96	99.68
LinearSVC	10^{-5}	0.0574	99.96	99.68
LinearSVC	10^{-6}	0.0774	99.96	99.68
LogisticRegression	10^{-2}	0.0122	94.17	93.44
LogisticRegression	10^{-3}	0.0382	99.28	98.04
LogisticRegression	10^{-4}	0.1106	99.64	98.68
LogisticRegression	10^{-5}	0.1798	99.64	98.68
LogisticRegression	10^{-6}	0.2294	99.64	98.68

Analysis: LinearSVC is robust to tolerance: even with $\text{tol} = 10^{-2}$, it achieves 99.36% test accuracy, and accuracy saturates at $\text{tol} \leq 10^{-3}$. In contrast, LogisticRegression is much more sensitive to tolerance — at $\text{tol} = 10^{-2}$, it only reaches 93.44% test accuracy, requiring $\text{tol} \leq 10^{-4}$ for convergence. Training time for LogisticRegression scales approximately linearly with $1/\text{tol}$, while LinearSVC remains nearly constant.

3.4 Experiment D: Effect of Penalty (L1 vs L2)

Table 4: Effect of penalty (regularization type) on LinearSVC and LogisticRegression ($C = 1.0$).

Model	Penalty	Training Time (s)	Train Acc (%)	Test Acc (%)
LinearSVC	L1	3.5301	100.00	99.72
LinearSVC	L2	0.0762	99.96	99.68
LogisticRegression	L1	14.0194	99.76	98.84
LogisticRegression	L2	0.1216	99.64	98.68

Analysis: L1 regularization produces slightly higher accuracy (sparsity may help identify the most relevant cross-product features), but is dramatically slower: **46** $\times$ slower for LinearSVC and **115** $\times$ slower for LogisticRegression. Given the marginal accuracy improvement ($<0.2\%$), L2 is the clear winner for the speed–accuracy tradeoff.

3.5 Summary of Hyperparameter Findings

1. LinearSVC with squared_hinge loss and dual=False offers the best speed–accuracy tradeoff.
2. $C = 1.0$ is optimal: sufficiently large to separate the data but not so large as to cause slow convergence.
3. Tolerance has minimal impact on LinearSVC but significantly affects LogisticRegression.
4. L2 penalty is strongly preferred over L1 due to orders-of-magnitude faster training.
5. LinearSVC consistently outperforms LogisticRegression in both speed and accuracy on this task.

4 Training Data Efficiency (Part 5)

To determine the minimum number of Challenge-Response Pairs (CRPs) required to successfully break the Hamming PUF, we trained the LinearSVC model on subsets of the public training data ranging from 100 to 7500 points. The test set was held constant at 2500 CRPs for all trials. For each training set size, we ran 5 independent trials with different random subsets and report the mean and standard deviation.

288-dimensional hyperplane and achieve near-perfect prediction. Based on the experimental trials, the minimum number of training CRPs needed to cross the specified accuracy thresholds are:

- **95% Test Accuracy:** This threshold is crossed at approximately **2,000 training CRPs**, where the model achieves $95.51\% \pm 0.45\%$ accuracy.
- **97% Test Accuracy:** This threshold is crossed at approximately **3,000 training CRPs**, where the model achieves $97.70\% \pm 0.18\%$ accuracy.
- **99% Test Accuracy:** This threshold is crossed at approximately **4,500 training CRPs**, where the model achieves $99.08\% \pm 0.04\%$ accuracy.

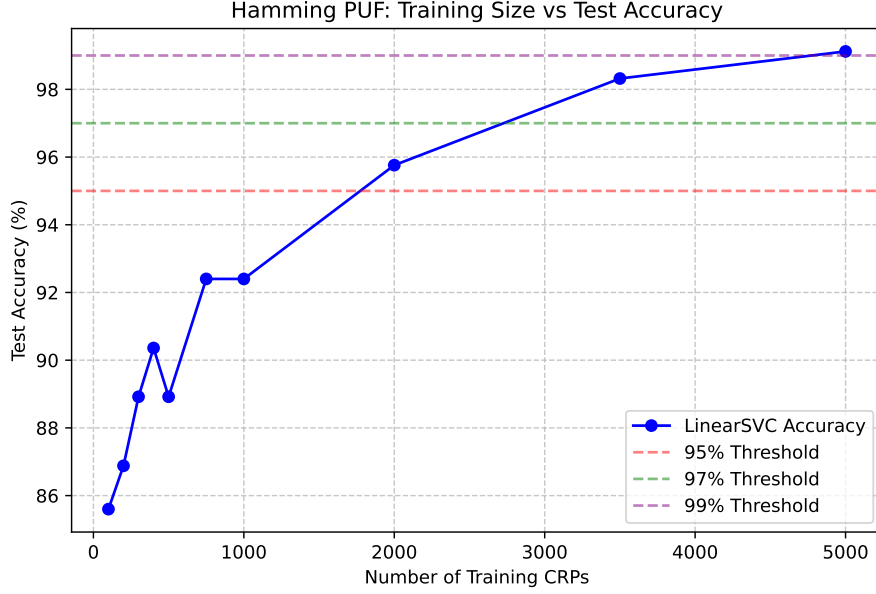


Figure 1: Test accuracy of LinearSVC across varying training set sizes. Error bars show ± 1 standard deviation over 5 random trials. Horizontal lines indicate 95%, 97%, and 99% accuracy thresholds.

Observations. The learning curve exhibits a characteristic log-linear pattern: accuracy rises rapidly from 82% to 95% between 100–2000 CRPs, then shows diminishing returns beyond 5000 CRPs. The standard deviation decreases with training size, indicating more stable models with more data. The model achieves approximately 288 effective parameters (the dimensionality of ϕ), suggesting that a rule of thumb of $\sim 7 \times D$ training points suffices for $>95\%$ accuracy and $\sim 15 \times D$ for $>99\%$ accuracy.